

Міністерство освіти і науки України
Національний університет «Запорізька Політехніка»

Кафедра програмних засобів

ЗВІТ

з самостійної роботи №1

з дисципліни «Системний аналіз» на тему:

«Застосування системного підходу до програмування мовою Python»

Виконав

Студент групи КНТ-122

О. А. Онищенко

Прийняли

Викладач

В. М. Льовкін

2024

ЗАСТОСУВАННЯ СИСТЕМНОГО ПІДХОДУ ДО ПРОГРАМУВАННЯ МОВОЮ PYTHON

Мета роботи

Навчитися застосовувати системний підхід до розробки програмного забезпечення мовою Python та набути практичних навичок використання інструментів мови Python.

Перша робота

Завдання до роботи

- Файл містить перелік повних адрес файлів (ім'я диску, список каталогів, ім'я файлу та розширення). Виділити з кожної адреси ім'я файлу, розширення та адресу першого каталогу. Перевірити для кожного файлу чи існує він. Вивести у файл, ім'я якого формується з імені початкового файлу додаванням постфіксу "str", перелік файлів, які існують на диску, згрупувавши їх за форматами та відсортувавши в алфавітному порядку за іменем файлів. Імена файлів виводити у форматі "/першийКаталог/.../ім'яФайлу/", сортуючи за розширенням та шляхом

- Продаж квитків у кінотеатр з можливістю переглядати сеанси, переглядати доступні та зайняті місця для перегляду заданого сеансу у відповідному залі, бронювання та звільнення місць. Інформація про нові сеанси може додаватися

Код програми

```
# main.py  
  
import json  
import random
```

```

import inquirer
from os import path
from rich import box
from rich.table import Table
from rich.console import Console
from rich.traceback import install
from collections import defaultdict

install()
console = Console()

def taskOne() -> None:
    currentDir: str = path.dirname(path.abspath(__file__))
    inputFilePath: str = path.join(currentDir, "..", "data", "files.txt")

    doUseReadyFile: str = inquirer.prompt(
        [
            inquirer.List(
                "choice",
                message="Would you like to use a predefined example or
enter your own file path?",
                choices=["Predefined Example", "Own File Path"],
            )
        ]
    )["choice"]
    if doUseReadyFile == "Own File Path":
        inputFilePath = inquirer.prompt(
            [
                inquirer.Text(
                    "file path",
                    message="Enter your input file path",
                    validate=lambda _, x: "\\" in x or "/" in x and x !=
"",
                )
            ]
        )["file path"]

    outputFilePath: str = path.join(
        currentDir, "..", "data", inputFilePath.split("\\")[-1][:-4] +
        "_str.txt"
    )

    with open(inputFilePath, "r", encoding="utf-8") as f:
        fileNames: list[str] = [line.strip() for line in f.readlines()]

    filesData: list[dict] = [
        {
            "name": file.split("\\")[-1].split(".")[0],

```

```

        "extension": file.split("\\")[-1].split(".")[-1],
        "first_dir": file.split("\\")[1],
        "does_exist": path.exists(file),
        "full_path": file,
    }
    for file in fileNames
]

outputTable: Table = Table(box=box.ROUNDED, title="All Files")
outputTable.add_column("Index", justify="right", style="cyan",
no_wrap=True)
outputTable.add_column("File Name", style="green")
outputTable.add_column("Extension", style="blue")
outputTable.add_column("First Directory", style="magenta")
outputTable.add_column("Does Exist", style="red", no_wrap=True,
justify="right")
for i, file in enumerate(filesData):
    outputTable.add_row(
        f"{i}",
        file["name"],
        file["extension"],
        file["first_dir"],
        "[green]Yes[/]" if file["does_exist"] else "No",
    )
console.print("\n", outputTable, "\n")

with console.status("Checking for existing files...",
spinner="point"):
    existingFiles: list[dict] = [file for file in filesData if
file["does_exist"]]
    existingFiles.sort(key=lambda x: (x["extension"], x["full_path"],
x["name"]))

    if len(existingFiles) == 0:
        console.print(
            "[red]No existing files found. Please check your input file
path or contents.[/]\n"
        )
        return

resultsTable: Table = Table(box=box.ROUNDED, title="Existing Files")
resultsTable.add_column("Index", justify="right", style="cyan",
no_wrap=True)
resultsTable.add_column("File Name", style="green")
resultsTable.add_column("Extension", style="blue")
resultsTable.add_column("File Path", style="yellow")
for i, file in enumerate(existingFiles):
    resultsTable.add_row(

```

```

        f"{i}",
        file["name"],
        file["extension"],
        f"/{file['first_dir']}/.../{file['name']}.{file['extension']}"
    ),
    console.print(resultsTable, "\n")

    with open(outputFilePath, "w", encoding="utf-8") as f:
        prevExtension: str = ""
        for file in existingFiles:
            curExtension: str = file["extension"]
            if curExtension != prevExtension:
                f.write(f"\n{curExtension.upper()}\n")
            f.write(f"{file['name']}.{file['extension']}\n")
            prevExtension = curExtension

def taskTwo() -> None:
    class Room:
        def __init__(self, number: int = 0, seats: list[list[int]] = [])
        -> None:
            self.seats: list[list[int]] = seats
            self.number: int = number

    class Movie:
        def __init__(self, title: str = "Movie", rooms: list[Room] = [])
        -> None:
            self.title: str = title
            self.rooms: list[Room] = rooms

    class Ticket:
        def __init__(self, movie: Movie, room: Room, row: int, seat: int)
        -> None:
            self.movie: Movie = movie
            self.room: Room = room
            self.row: int = row
            self.seat: int = seat

    class Cinema:
        def __init__(self, name: str = "Cinema", movies: list[Movie] =
        []) -> None:
            self.movies: list[Movie] = movies
            self.name: str = name
            self.tickets: list[Ticket] = []
            self.watched: dict = defaultdict(int)

        def buyTicket(self, movie: Movie, room: Room, row: int, seat:
        int) -> None:

```

```

        room.seats[row][seat] = 0
        self.tickets.append(Ticket(movie, room, row, seat))

    def sellTicket(self, ticket: Ticket) -> None:
        ticket.room.seats[ticket.row][ticket.seat] = 1
        self.tickets.remove(ticket)

    def readMoviesFromJson(self, path: str = "") -> None:
        with open(path, "r", encoding="utf-8") as f:
            data = json.load(f)
            self.name = data["name"]
            for movieData in data["movies"]:
                rooms = []
                for room in movieData["rooms"]:
                    rooms.append(Room(number=room["number"],
seats=room["seats"]))
                self.movies.append(Movie(title=movieData["title"],
rooms=rooms))

    def drawSeatsGrid(room: Room, movie: Movie) -> None:
        seats: list[list[str]] = [
            ["⬤" if not seat else "●" for seat in row] for row in
room.seats
        ]

        table: Table = Table.grid(padding=(1, 1))
        for i, row in enumerate(seats):
            table.add_row(*[f"[bold]{i+1}[/]" + row])

        console.print(
            f'[bold]All seats for "{movie.title}" in room'
            f'#{room.number}[/bold]\n',
            table,
            "\n",
        )

    def drawTicketsTable(tickets: list[Ticket]) -> None:
        table: Table = Table(box=box.ROUNDED, title="Bought Tickets")
        table.add_column("Index", justify="right", style="cyan",
no_wrap=True)
        table.add_column("Movie", style="bold yellow")
        table.add_column("Year", style="violet", no_wrap=True)
        table.add_column("Room", style="magenta", no_wrap=True)
        table.add_column("Row", style="green", no_wrap=True)
        table.add_column("Seat", style="blue", no_wrap=True)
        for i, ticket in enumerate(tickets):
            name, year = ticket.movie.title.split(" - ")
            table.add_row(

```

```

        f"{i+1}",
        f"{name}",
        f"{year}",
        f"{ticket.room.number}",
        f"{ticket.row+1}",
        f"{ticket.seat+1}",
    )
    console.print(table, "\n")

def drawMoviesTable() -> None:
    table: Table = Table(box=box.ROUNDED, title="Watched Movies")
    table.add_column("Index", style="cyan", no_wrap=True,
justify="right")
    table.add_column("Movie", style="bold yellow", no_wrap=True)
    table.add_column("Year", style="violet", no_wrap=True)
    table.add_column("Times Watched", style="magenta", no_wrap=True,
justify="left")
    for i, (movie, count) in enumerate(cinema.watched.items()):
        name, year = movie.split(" - ")
        table.add_row(
            f"{i+1}",
            f"{name}",
            f"{year}",
            f"{count}",
        )
    console.print(table, "\n")

currentDir: str = path.dirname(path.abspath(__file__))
moviesDataPath: str = path.join(currentDir, "..", "data",
"movies.json")

cinema: Cinema = Cinema()
cinema.readMoviesFromJson(moviesDataPath)

while True:
    actions: list[str] = [
        "Browse Tickets",
        "View Bought Tickets",
        "View Watched Movies",
        "Add New Movie",
        "Exit",
    ]
    action: str = inquirer.prompt(
        [
            inquirer.List(
                "action",
                message="What would you like to do?",
                choices=actions,

```

```

        )
    ]
)["action"]

if action == "Browse Tickets":
    if not cinema.movies:
        console.print("[bold]No movies available[/bold]\n")

    movie: str = inquirer.prompt(
        [
            inquirer.List(
                "movie",
                message="Which movie would you like to watch?",
                choices=[movie.title for movie in cinema.movies],
            )
        ]
    )["movie"]
    movie: Movie = [movie.title for movie in
cinema.movies].index(movie)
    movie: Movie = cinema.movies[movie]

    room: int = inquirer.prompt(
        [
            inquirer.List(
                "room",
                message="Choose a room (hall) where you would
like to watch the movie",
                choices=range(1, len(movie.rooms) + 1),
            )
        ]
    )["room"]
    room: Room = movie.rooms[room - 1]

    drawSeatsGrid(room, movie)

    availableRows: list = [
        i + 1 for i, row in enumerate(room.seats) if True in row
    ]
    row: int = inquirer.prompt(
        [
            inquirer.List(
                "row",
                message="Choose a seat row where you would like
to watch the movie",
                choices=availableRows,
            )
        ]
    )["row"]

```



```

        row: int = row - 1

        seats: list[int] = [i + 1 for i, seat in
enumerate(room.seats[row]) if seat]
        seat: int = inquirer.prompt(
            [
                inquirer.List(
                    "seat",
                    message="Choose a seat where you would like to
watch the movie",
                    choices=seats,
                )
            ]
        )["seat"]
        seat: int = seat - 1

        confirmation: dict = inquirer.prompt(
            [
                inquirer.Confirm(
                    "confirm",
                    message=f'Are you sure you want to buy a ticket
for "{movie.title}" in room {room.number} row {row+1} seat {seat+1}?',
                    default=True,
                )
            ]
        )

        if confirmation["confirm"]:
            cinema.buyTicket(movie, room, row, seat)
            console.print("\n[bold]Ticket bought![/bold]\n")

        drawSeatsGrid(room, movie)

    elif action == "View Bought Tickets":
        if not cinema.tickets:
            console.print("[bold]No bought tickets found[/bold]\n")
            continue
        drawTicketsTable(cinema.tickets)

        indices: list = range(1, len(cinema.tickets) + 1)
        ticketIndex: int = inquirer.prompt(
            [
                inquirer.List(
                    "ticket",
                    message="Choose a ticket number",
                    choices=indices,
                )
            ]
        )

```

```

) ["ticket"]
ticketIndex: int = ticketIndex - 1

action: str = inquirer.prompt(
    [
        inquirer.List(
            "action",
            message="What would you like to do?",
            choices=["Watch movie", "Return ticket", "Exit"],
        )
    ]
) ["action"]

ticketData: Ticket = cinema.tickets[ticketIndex]
movieName: str = ticketData.movie.title

if action == "Return ticket":
    cinema.sellTicket(ticketData)
    console.print(f'[bold]"{movieName}"[/bold] was
returned!\n')
elif action == "Watch movie":
    cinema.watched[movieName] += 1
    console.print(f'[bold]"{movieName}"[/bold] was
watched!\n')
    cinema.tickets.remove(ticketData)

elif action == "View Watched Movies":
    if not cinema.watched:
        console.print("[bold]No watched movies found[/bold]\n")
        continue
    drawMoviesTable()

elif action == "Add New Movie":
    title: str = inquirer.prompt(
        [
            inquirer.Text(
                "title",
                message="Enter movie title. Make sure to separate
the title from the year with a dash (-)",
                validate=lambda _, x: x != "" and "-" in x,
            )
        ]
    ) ["title"]
    title, year = title.split("-")
    title, year = title.strip(), year.strip()

numOfRooms: int = inquirer.prompt(
    [

```

```

        inquirer.Text(
            "rooms",
            message="Enter number of available rooms",
            validate=lambda _, x: x != "" and x.isdigit(),
        )
    ]
)["rooms"]
numOfRooms: int = int(numOfRooms)

seats: list[list[list[int]]] = [
    [[random.choice([0, 1]) for _ in range(10)] for _ in
range(5)]
    for _ in range(numOfRooms)
]
rooms: list[Room] = []
for i in range(1, numOfRooms + 1):
    rooms.append(Room(i, seats[i - 1]))

console.print(f'\n[bold]"{title} - {year}"[/bold] has been
added!\n')
movie: Movie = Movie(f"{title} - {year}", rooms)
cinema.movies.append(movie)

else:
    break

def main() -> None:
    availableTasks = [
        "First - Checking Files",
        "Second - Cinema Tickets",
    ]
    selectedTask: str = inquirer.prompt(
        [
            inquirer.List(
                "task",
                message="Which task would you like to look at?",
                choices=availableTasks,
            )
        ]
    )["task"]

    if selectedTask == availableTasks[0]:
        taskOne()
    elif selectedTask == availableTasks[1]:
        taskTwo()

if __name__ == "__main__":

```

```
main()
```

Результати виконання

```
[?] Would you like to use a predefined example or enter your own file path?: Predefined Example
> Predefined Example
Own File Path
```

All Files

Index	File Name	Extension	First Directory	Does Exist
0	definitely_doesnt	txt	everything	Yes
1	definitely_does	txt	everything	No
2	does_it	txt	everything	Yes
3	file	txt	code	No
4	doesnt_it	txt	everything	No
5	exists	txt	everything	Yes
6	maybe_not	txt	everything	No
7	maybe	txt	everything	Yes
8	file	json	play	No
9	not_sure	txt	everything	Yes
10	am_sure	txt	everything	No
11	exist	md	files	No
12	perhaps	txt	everything	Yes
13	or_not	txt	everything	No
14	should_check	txt	everything	Yes
15	main	py	code	No
16	should_exist	txt	everything	No
17	probably	json	everything	Yes
18	main	json	warehouse	No
19	in_theory_exists	txt	everything	No
20	maybe	md	everything	Yes

Existing Files

Index	File Name	Extension	File Path
0	probably	json	/everything/.../probably.json
1	maybe	md	/everything/.../maybe.md
2	definitely_doesnt	txt	/everything/.../definitely_doesnt.txt
3	does_it	txt	/everything/.../does_it.txt
4	exists	txt	/everything/.../exists.txt
5	maybe	txt	/everything/.../maybe.txt
6	not_sure	txt	/everything/.../not_sure.txt
7	perhaps	txt	/everything/.../perhaps.txt
8	should_check	txt	/everything/.../should_check.txt

Рисунок 1.1 – Результати роботи програми для завдання один

При виконанні завдання як вхідні дані був використаний файл наступного змісту:

```
1 D:\everything\University\y2t2\SA\tasks\lb1\dev\data\existingFiles\definitely_doesnt.txt
2 D:\everything\University\y2t2\SA\tasks\lb1\dev\data\existingFiles\definitely_does.txt
3 D:\everything\University\y2t2\SA\tasks\lb1\dev\data\existingFiles\does_it.txt
4 D:\code\first_project\file.txt
5 D:\everything\University\y2t2\SA\tasks\lb1\dev\data\existingFiles\doesnt_it.txt
6 D:\everything\University\y2t2\SA\tasks\lb1\dev\data\existingFiles\exists.txt
7 D:\everything\University\y2t2\SA\tasks\lb1\dev\data\existingFiles\maybe_not.txt
8 D:\everything\University\y2t2\SA\tasks\lb1\dev\data\existingFiles\maybe.txt
9 C:\play\one\three\two\file.json
10 D:\everything\University\y2t2\SA\tasks\lb1\dev\data\existingFiles\not_sure.txt
11 D:\everything\University\y2t2\SA\tasks\lb1\dev\data\existingFiles\am_sure.txt
12 D:\files\directory\exist.md
13 D:\everything\University\y2t2\SA\tasks\lb1\dev\data\existingFiles\perhaps.txt
14 D:\everything\University\y2t2\SA\tasks\lb1\dev\data\existingFiles\or_not.txt
15 D:\everything\University\y2t2\SA\tasks\lb1\dev\data\existingFiles\should_check.txt
16 C:\code\first_project\main.py
17 D:\everything\University\y2t2\SA\tasks\lb1\dev\data\existingFiles\should_exist.txt
18 D:\everything\University\y2t2\SA\tasks\lb1\dev\data\existingFiles\probably.json
19 D:\warehouse\data\projects\first_project\main.json
20 D:\everything\University\y2t2\SA\tasks\lb1\dev\data\existingFiles\in_theory_exists.txt
21 D:\everything\University\y2t2\SA\tasks\lb1\dev\data\existingFiles\maybe.md
```

Рисунок 1.2 – Зміст файлу з вхідними даними

На виході в результаті роботи програми було отримано файл наступного змісту:

```
1  JSON
2  probably.json
3
4  MD
5  maybe.md
6
7  TXT
8  definitely_doesnt.txt
9  does_it.txt
10 exists.txt
11 maybe.txt
12 not_sure.txt
13 perhaps.txt
14 should_check.txt
15
```

Рисунок 1.3 – Вихідний файл з результатами роботи програми

Результати виконання програми завдання два наведені нижче у вигляді знімків з екрану:

```
[?] What would you like to do?: Browse Tickets
> Browse Tickets
View Bought Tickets
View Watched Movies
Add New Movie
Exit

[?] Which movie would you like to watch?: The Nativity Story - 2006
Jesus - 1979
> The Nativity Story - 2006
The Passion of the Christ - 1988
Paul, Apostle of Christ - 2018
Пропала грамота - 1972
Поводир - 2014
Мавка. Лісова пісня - 2023
Клондайк - 2022

[?] Choose a room (hall) where you would like to watch the movie: 2
1
> 2

All seats for "The Nativity Story - 2006" in room #2

1 ● ● ● ● ● ● ● ● ● ●
2 ● ● ● ● ● ● ● ● ● ●
3 ● ● ● ● ● ● ● ● ● ●
4 ● ● ● ● ● ● ● ● ● ●
5 ● ● ● ● ● ● ● ● ● ●

[?] Choose a seat row where you would like to watch the movie: 3
1
2
> 3
4
5

[?] Choose a seat where you would like to watch the movie: 4
1
> 4
6
9
10

[?] Are you sure you want to buy a ticket for "The Nativity Story - 2006" in room 2 row 3 seat 4? (Y/n):
Ticket bought!
```

```
[?] What would you like to do?: View Bought Tickets
  Browse Tickets
> View Bought Tickets
  View Watched Movies
  Add New Movie
  Exit
```

Bought Tickets

Index	Movie	Year	Room	Row	Seat
1	The Nativity Story	2006	2	3	4

```
[?] Choose a ticket number: 1
> 1
```

```
[?] What would you like to do?: Watch movie
> Watch movie
  Return ticket
  Exit
```

"The Nativity Story – 2006" was watched!

```
[?] What would you like to do?: View Watched Movies
  Browse Tickets
  View Bought Tickets
> View Watched Movies
  Add New Movie
  Exit
```

Watched Movies

Index	Movie	Year	Times Watched
1	The Nativity Story	2006	1


```

[?] What would you like to do?: Add New Movie
  Browse Tickets
  View Bought Tickets
  View Watched Movies
> Add New Movie
  Exit

[?] Enter movie title. Make sure to separate the title from the year with a dash (-): Movie Name - 2024
[?] Enter number of available rooms: 7

"Movie Name - 2024" has been added!

[?] What would you like to do?: Browse Tickets
> Browse Tickets
  View Bought Tickets
  View Watched Movies
  Add New Movie
  Exit

[?] Which movie would you like to watch?: Movie Name - 2024
  Jesus - 1979
  The Nativity Story - 2006
  The Passion of the Christ - 1988
  Paul, Apostle of Christ - 2018
  Пропала грамота - 1972
  Поводир - 2014
  Мавка. Лісова пісня - 2023
  Клондайк - 2022
> Movie Name - 2024

[?] Choose a room (hall) where you would like to watch the movie: 7
  1
  2
  3
  4
  5
  6
> 7

All seats for "Movie Name - 2024" in room #7

1 ● ● ● ● ● ● ● ● ● ●
2 ● ● ● ● ● ● ● ● ● ●
3 ● ● ● ● ● ● ● ● ● ●
4 ● ● ● ● ● ● ● ● ● ●
5 ● ● ● ● ● ● ● ● ● ●

```

Як вхідні дані використовувався файл JSON наступного змісту:

```

1  {
2    "name": "Кінотеатр ім. Довженка",
3    "movies": [
4      {
5        "title": "Jesus - 1979",
6        "rooms": [
7          {
8            "number": 1,
9            "seats": [
10             [0, 1, 0, 0, 0, 1, 1, 1, 1, 1],
11             [1, 1, 1, 0, 1, 0, 1, 0, 1, 0],
12             [1, 1, 0, 0, 0, 1, 1, 0, 0, 1],
13             [1, 1, 1, 0, 0, 1, 1, 0, 0, 0],
14             [0, 1, 1, 1, 0, 1, 1, 1, 0, 1]
15           ]
16         },
17         {
18           "number": 2,
19           "seats": [
20             [1, 1, 1, 1, 1, 1, 1, 0, 0, 1],
21             [1, 0, 1, 1, 0, 1, 0, 0, 1, 1],
22             [1, 1, 1, 1, 1, 0, 0, 1, 0, 0],
23             [0, 1, 0, 1, 0, 1, 1, 0, 0, 1],
24             [1, 1, 1, 0, 0, 1, 0, 1, 1, 1]
25           ]
26         }
27       ]
28     },
29     ...
30   ]
31 }

```

Рисунок 1.5 – Файл вхідних даних для програми завдання два

Друга робота

Завдання до роботи

- Вивести всі коректні комбінації пар круглих дужок, які можна сформувати з n дужок, що закриваються і відкриваються. Наприклад, коректна комбінація $((()))$, некоректна $((()))($. Кількість дужок задається користувачем

- Визначити кількість слів у тексті, що зберігається у файлі, та довжину найкоротшого слова. Слова відділяються одне від одного не тільки пробілами, але й будь-якими знаками пунктуації

Код програми

```
# main.py

import inquirer
from rich.console import Console
from rich.traceback import install

import one as taskOne
import two as taskTwo

install()
console = Console()

tasks: dict[str, callable] = {
    "First - Bracket Pairs": taskOne.main,
    "Second - Words Counter": taskTwo.main,
}

def main() -> None:
    availableTasks: list[str] = list(tasks.keys())
    selectedTask: str = inquirer.prompt(
        [
            inquirer.List(
                "task",
                message="Which task would you like to look at?",
```

```

        choices=availableTasks,
    )
]
)["task"]

tasks[selectedTask]()

if __name__ == "__main__":
    main()

# one.py

import inquirer
from rich.console import Console
from rich.traceback import install

install()
console = Console()

def functional(n: int) -> None:
    def helper(openCount: int, closeCount: int, currentCombination:
list[str]):
        def printCombination() -> None:
            console.print("".join(currentCombination))

        def addLeftParenthesis() -> None:
            currentCombination.append("(")
            helper(openCount + 1, closeCount, currentCombination)
            currentCombination.pop()

        def addRightParenthesis() -> None:
            currentCombination.append(")")
            helper(openCount, closeCount + 1, currentCombination)
            currentCombination.pop()

        (openCount == closeCount == n and printCombination()) or (
            openCount < n and addLeftParenthesis()
        ) or (closeCount < openCount and addRightParenthesis())

    helper(0, 0, [])

def imperative(n: int) -> None:
    stack: list[tuple[str, int, int]] = []
    stack.append(("", 0, 0))

```

```

while stack:
    currentCombination, openCount, closeCount = stack.pop()

    def printCombination() -> None:
        console.print(currentCombination)

    def addLeftParenthesis() -> None:
        stack.append((currentCombination + "(", openCount + 1,
closeCount))

    def addRightParenthesis() -> None:
        stack.append((currentCombination + ")", openCount, closeCount
+ 1))

    (openCount == closeCount == n and printCombination()) or (
        openCount < n and addLeftParenthesis()
    ) or (closeCount < openCount and addRightParenthesis())

def main() -> None:
    tasks: dict[str, callable] = {
        "Functional Programming": functional,
        "Imperative Programming": imperative,
    }
    choice: str = inquirer.prompt(
        [
            inquirer.List(
                "solution",
                message="Which solution would you like to use?",
                choices=list(tasks.keys()),
            )
        ]
    )["solution"]

    bracketsNumber: str = inquirer.prompt(
        [
            inquirer.Text(
                "number of brackets",
                message="How many bracket pairs would you like to have?",
                validate=lambda _, x: x != "" and x.isdigit() and int(x)
> 0,
            )
        ]
    )["number of brackets"]
    bracketsNumber: int = int(bracketsNumber)

    console.print(f"\nAll possible bracket pairs for {bracketsNumber}
bracket pairs:")
    tasks[choice](n=bracketsNumber)

```

```

if __name__ == "__main__":
    main()

# two.py

import inquirer
from os import path
from functools import reduce
from rich.console import Console
from rich.traceback import install

install()
console = Console()

def functional(filePath: str) -> tuple[int, int]:
    with open(filePath, "r", encoding="utf-8") as file:
        text = file.read()

    punctuation: str = '!@#$%^&*()_+{|}: "<>?`~'
    table: str = str.maketrans("", "", punctuation)

    words: list[str] = text.translate(table).split()
    wordsCount: int = len(words)

    shortestWordLength: int = reduce(
        lambda shortestSoFar, word: min(shortestSoFar, len(word)), words,
        float("inf")
    )

    return wordsCount, shortestWordLength

def main() -> None:
    def getOwnFilePath() -> str:
        filePath = inquirer.prompt(
            [
                inquirer.Text(
                    name="file path",
                    message="Enter full path to your file",
                    validate=lambda _, x: path.exists(x) and
path.isfile(x),
                )
            ]
        )["file path"]

```

```

    return filePath

currentDir: str = path.dirname(path.abspath(__file__))
filePath: str = path.join(currentDir, "..", "data", "input.txt")

choices: list[str] = [
    "Use provided file",
    "Enter path for my own",
]
decision: str = inquirer.prompt(
    [
        inquirer.List(
            "choose",
            message="Which path would you like to use?",
            choices=choices,
        )
    ]
)["choose"]

(decision == choices[0] and filePath) or (
    decision == choices[1] and getOwnFilePath()
)

wordsCount, shortestWordLength = functional(filePath)
console.print(f"Number of words: {wordsCount}")
console.print(f"Length of shortest word: {shortestWordLength}")

if __name__ == "__main__":
    main()

```

Результати виконання

Перше завдання

Після виконання коду до першого завдання отримуємо наступні результати:

```
[?] Which solution would you like to use?: Functional Programming
> Functional Programming
Imperative Programming

[?] How many brackets would you like to use?: 3

All possible bracket pairs for 3 bracket pairs:
((( )))
(( ( )) )
(( )) ( )
( ( ( ) ) )
( ( ) ) ( )
( ) ( ( ) )
```

Рисунок 1.1 – Результати виконання першого завдання

Друге завдання

Під час виконання другого завдання як вхідний використався файл наступного змісту:

```
Святе Євангеліє від Івана 3:15-21

15 І тоді всі, хто вірить у Нього, одержать вічне життя. 16 Бо так
сильно полюбив Бог цей світ, що віддав Свого Єдиного Сина заради
того, щоб кожен, хто в Нього вірує, не був загублений, а здобув
вічне життя.

17 Не на осуд світу послав Бог Свого Сина, а на те, щоб світ через
Нього спасся. 18 Того, хто вірує в Нього, не буде засуджено, але
того, хто не вірить, вже засуджено, бо він не повірив у Єдиного
Сина Божого.

19 Ось підстава для засудження: Світло прийшло у світ, та люди
віддали перевагу темряві, бо лихими були вчинки їхні. 20 Кожен,
хто чинить зло, ненавидить Світло й намагається уникати його, щоб
вчинки його лишалися таємними. 21 Той же, хто йде за правдою,
прямує до Світла, щоб видно було, що вчинки його від Бога».
```

Рисунок 1.2 – Зміст файлу вхідних даних для другого завдання

Після виконання коду до другого завдання маємо наступний результат:

```
[?] Which path would you like to use?: Use provided file  
> Use provided file  
Enter path for my own  
  
Number of words: 138  
Length of shortest word: 1
```

Рисунок 1.3 – Результати виконання другого завдання

Висновки

Таким чином, ми навчилися застосовувати системний підхід до розробки програмного забезпечення мовою Python та набули практичних навичок використання інструментів мови Python.